
VNS Mission Console

Vision Navigation Analysis System

Project Report

Everything built, changed and shipped across the project —
compiled from CHAT_LOG.md (the live session/continuation log)

Date: 6 October 2026

Source: CHAT_LOG.md | Project root: C:\YousufVNS\vns-app\vns-app

Contents

1. Project Overview & Architecture	3
2. Getting Started	3
3. Pipeline Engines (10 stages)	4
4. Auth & Backend (admin approval, password recovery)	5
5. Report Generation System	5
6. UI / UX Features & Changes	7
7. Feature Status	8
8. Packaging, Installer & Delivery	8
9. Known Issues & Gotchas	9
10. Current Work State & Recent Changes	9

Project Overview & Architecture

What it is

The VNS (Vision Navigation Analysis) Mission Console is a desktop mission-control application for a vision-guided rover. A React + Electron UI provides the operator screens; an Express backend handles authentication and the admin approval flow; and a set of pure-Python pipeline engines implement every image/vision stage. Each engine is compiled to a single .exe so deployed machines do not need Python. Report PDFs are generated by a hand-written, pure-stdlib PDF writer in the pipeline.

Since §27 the whole stack ships as ONE portable executable: the React build, the Express backend (running inside the Electron main process) and an embedded SQLite database are all packed into release\vns-app 0.0.0.exe (~96 MB). No installer, no MySQL, no Node, no Python on the target machine — double-click and it runs, fully offline.

Repository layout

electron/main.cjs — Electron main process: IPC, workspace folders, pipeline runner, save-dialog, report wiring.

electron/preload.cjs — renderer bridge (window.workspace) into the main process.

electron/scriptRunner.cjs — ScriptRunner + SCRIPT_CONFIG (10 stages incl. report), .exe override resolution.

electron/reportEngine.cjs — assembles the report JSON manifest from data-URL images and runs the report stage.

scripts/pipeline/*.py — 10 stdlib-only stage scripts + the vnsio.py helper + report.py PDF writer.

scripts/bin/ — compiled single-file .exe per stage (auto-preferred over .py by ScriptRunner).

scripts/build_pipeline_exe.ps1 — PyInstaller builder used to compile every stage.

electron/server/ — embedded backend (§19/§27): app.cjs, db.cjs (better-sqlite3 schema + admin seed), routes for auth/admin/role-requests — runs inside the Electron process on 127.0.0.1:4000.

src/ — React renderer: VNSApp.jsx (all screens), hooks/useWorkspace.js (IPC hooks), auth/, App.css, CustomScreens.jsx.

server/ — the original external Express API + MySQL schema; kept in the repo but UNUSED since §27 (superseded by electron/server/).

start-dev.ps1 / stop-dev.ps1 — one-command start/stop; since §27 they launch only Vite + Electron (the backend rides along inside the app, no MySQL).

CHAT_LOG.md — the session/continuation log this report is generated from, with the full verbatim history.

Key architectural decisions

Pure stdlib pipeline scripts (no numpy/opencv/reportlab) so every stage can be PyInstaller-packaged to one .exe.

Stage executables follow an EXECUTABLE CONTRACT: stage.exe [inputImagePath?] [outputDir], cwd = workspace root, exit 0 = success.

No pdfkit/reportlab: the PDF report is written pixel-by-pixel (object + xref table assembled by hand), images embedded as DCTDecode (JPEG) or rebuilt PNG scanlines.

Workspace/session folder is intentionally NOT persisted across app restarts — fresh launch starts empty and the user picks the folder again.

Getting Started

Everything is scripted. From the project root:

Command	What it does
start-dev.ps1	Launches Vite + Electron (embedded backend starts with the app)
stop-dev.ps1	Stops the app
npm run electron:build	Builds dist + packages with electron-builder (portable target)
python make_project_report.py	Regenerates this PDF from CHAT_LOG.md

Admin login: admin@vns.local / VNSProject (seeded automatically into the database on first run)

Embedded backend: http://localhost:4000 (/api/health), bound to 127.0.0.1 — never exposed to the network.

Database: %APPDATA%\vns-app\vns.db — plain, unencrypted SQLite, created + schema'd + seeded on first launch.

Deployment: copy release\vns-app 0.0.0.exe to the target PC and double-click — no installer, no admin rights, no Node/Python/MySQL. First launch takes ~30 s (self-extraction to %TEMP%).

Accounts are per-machine (each install has its own database file) — the inherent trade-off of a single-system deployment.

Pipeline Engines (10 stages)

Ten pipeline stages are registered in `SCRIPT_CONFIG` inside `electron/scriptRunner.cjs`. Each maps a UI tab to a script (and a compiled .exe), and persists its outputs into a named stage folder inside the active session:

UI tab / stage	Script	Output folder
Input / preprocess	preprocess.py	04_Preprocessed
Obs. Det. / obstacleDetection	obstacle_detection.py	05_Obstacle.Detection
Dist. Map / distanceMap	distance_map.py	06_Distance_Map
Data / occupancyGrid	occupancy_grid.py	07_Occupancy_Grid
Safe Path / safePath	safe_path.py	08_Pred_Safe_Path
Navigation / navigation	navigation.py	09_Navigation
Telemetry / telecommand	telemetry.py (channel)	11_Telecommand_Data
Telemetry / telemetry	telemetry.py (channel)	12_Telemetry_Data
Health / roverHealth	rover_health.py	13_Rover_Health_Status
Scene Analysis / sceneAnalysis	scene_analysis.py	14_Scene_Analysis_Report
Report / report	report.py	PDF (save dialog)

Running stages

Every tab has a Run button wired to its own script; per-window ? buttons re-run just that window's stage (added §20.2).

The topbar Run Algorithms button (kept that name) runs ALL tabs sequentially: input copy, preprocess, obstacleDetection, occupancyGrid, safePath, distanceMap, navigation, sceneAnalysis, telemetry x2, roverHealth, with a pass/fail summary toast (§22.5).

ScriptRunner spawns .py via python (Windows can't spawn .py directly) and .exe directly; 120 s

timeout; stdout/stderr shown in a console modal.

Compilation to .exe (§17)

All 10 stages compiled with PyInstaller 6.22.2 (Python 3.14.7) into single-file console exes under scripts/bin (~9.5 MB each).

Exe vs .py verified byte-identical on the real session: equal output counts, identical PNG/JPGs, same MD5 PDF for report.exe.

Delivery swap: real production exes can be dropped in <workspace root>\bin\ — ScriptRunner checks that folder first (override), then the bundled app.asar.unpacked\scripts\bin, then the dev .py fallback. Zero code changes needed to swap a stage.

telemetry.py splits by channel so 11_Telecommand_Data (Img No/Module/Size/Value/Timestamp/Status) and 12_Telemetry_Data (VO Distance Traveled / Lunar Coordinates N/E) each get the exact schema the JS reader expects — fixed the 'no rows matched' spam (§17.3).

Auth & Backend (admin approval, password recovery)

Login, signup and accounts

Signup creates a pending account — no auto-login (status = pending until an admin acts).

Login blocks non-active accounts with 403 (pending / rejected).

users.status column added via live migration (pending / active / rejected).

Admin approval workflow (§7)

AdminPanel.jsx shows 'Pending New Accounts' with Approve / Reject and can grant viewer / editor / admin roles.

Endpoints: /api/admin/users/:id/approve and /reject.

The admin/bootstrap account is force-set active so the first login always works.

Password recovery (§7)

password_resets table + POST /api/auth/forgot-password (returns a dev_token since there is no mail service) + POST /api/auth/reset-password.

'Forgot password?' link + reset view in LoginForm.jsx — the LINK IS NOW HIDDEN in the UI (commented out by user request); no mention of the feature on the login screen. The backend endpoints and reset views remain in the code, reachable only via the API.

Verified end-to-end: reset works, the old password is rejected afterwards, and a used token cannot be reused.

Backend architecture: embedded SQLite (§19 -> §27)

The backend was first made self-contained in §19 (Express + better-sqlite3 embedded in Electron, MySQL dropped, DB at %APPDATA%\vns-app) as commit e7b182d, then REVERTED (3df2cdd) because a per-machine DB makes admin approval per-install instead of central. In §27 it was restored from git history and adopted as the SHIPPING architecture — the task requires a single-system deployment, where per-machine accounts are exactly right.

Electron's main process starts the Express app on 127.0.0.1:4000 at launch; db.cjs creates the schema (users incl. status, sessions, password_resets, role_requests) and seeds the admin on first run; the React renderer calls http://localhost:4000/api unchanged. The external server/ + portable MySQL setup remains in the repo but is no longer started.

Report Generation System

Report generation started life as pdfkit code in electron/main.cjs (buildReportPdf, drawReportTable). It now lives entirely in scripts/pipeline/report.py — a pure-stdlib PDF writer with no reportlab/pdfkit (reportlab has no wheels for Python 3.14.7; the project rule is stdlib-only so scripts package into one .exe).

How a report is generated (§16)

Renderer builds sections with base64 data-URL images — no renderer change when the layout changes.

electron/reportEngine.cjs materializes every image to a temp file, writes a JSON manifest {meta, sections:[{heading, intro, text, imageNotes, images:[{caption, path}], table}]}, and runs the registered report stage.

report.py reads the manifest and draws every pixel: cover page, numbered sections, figure grids with captions, and Parameter/Value (or User Query/AI Response) tables with a bordered grid.

reportEngine keeps the temp dir + stderr log if a run fails so failures are reproducible; the temp dir is cleaned up on success.

Bugs found and fixed while building it

16.1: exit code 1 — report failed on every real manifest. `_sanitize` remapped U+2014 (em dash) to `chr(0x97)`, which `str.encode('cp1252')` rejects. Now cp1252-encodable chars are kept as-is (U+2014 encodes to byte 0x97 that WinAnsi renders back as the em dash); only un-encodable chars are rewritten (arrow `->` `'->'`, else `'?'`).

16.2: rendered PDFs opened blank — an object-number collision made every page's `/Contents` point at a Page dictionary instead of a content stream. Content streams and page dicts now use separate number ranges.

16.3: strict viewers still blank / pypdf saw 0 text — the Pages tree `/Kids` was built from the content-stream numbers, so streams were referenced as pages. `/Kids` now names the page dicts, and fonts carry `/Encoding` / `WinAnsiEncoding` so text extracts cleanly.

16.4: centred text pushed half off-page — `align=center/right` used the box's right edge as the base. `x` is now the box's left edge and width extends rightwards (`center: x + (width-tw)/2`).

16.5: table cells misaligned/spilling and blank JPEG figures — each cell's baseline is now reset per cell, and `/DCTDecode` streams carry the raw JPEG bytes verbatim (no Flate layer around a JPEG SOI, which viewers could not decode).

Layout evolution (§23 -> §24)

§23 briefly moved the report to a US-Letter IEEE-paper style (title/abstract block, two columns, Roman headings, Fig./TABLE numbering, footer).

§24 superseded it: the user supplied 'Report Layout.docx' and wanted a ditto copy. The report now mirrors it exactly — A4 page with 1.25in side / 1in top+bottom margins and Times; a cover page ('Visison Navigation Analysis Report' / 'Image ID: <session>' / 'Date:', bold centred, then a hard break); numbered 1..9 bold headings; the intro sentence per section (the 'captured on <date>', at <time>' intro and the Left/Right camera captions keep the template's underlines); per-section figure grids ('Figure N: ...' centred under every image, numbering runs 1..25) with the template's grid shapes (full-width stacks, side-by-side pairs, preprocess = 2 pairs + full-width rectified, VO = pair + full-width localization); and Parameter/Value (or User Query/AI Response) tables with a 0.5pt grid, bold centred header, the first four sections keeping the template's two trailing empty rows.

Verified: 12 pages / 25 figures / 9 tables from a real session manifest; report.py and the rebuilt scripts/bin/report.exe produce byte-identical output (356649 bytes).

The Electron side needed NO change — generateFullReport already feeds the exact headings, intro sentences, captions and Property/QA tables.

Report UX

§13: the Data tab became a tab-level report summary — a 'Tabs to include in report' sidebar where ticking a tab includes that tab's sections.

§14: 'Generate Report' opens a native Save-As dialog (default 15_Report/<session>/), and cancelling shows an info toast instead of an error.

§21: sub-headings under each tab have checkboxes (default checked = included); unchecking one excludes exactly that caption/heading from the report.

UI / UX Features & Changes

Run buttons & pipeline UX

- §4: 'Run Detection' button on Obs. Det. with a console modal showing stdout/stderr + script/status/input/output paths, and friendly 'no workspace folder' handling.
- §4.4: fixed a blank Data tab (handler name mismatch + a stray runResult block that didn't belong there).
- §9: every core tab got its own Run button (navigation.py and scene_analysis.py are new scripts), via the reusable PipelineRunButton.
- §20.2: per-window triangular ? run buttons — each panel runs just its own stage (preprocess, obstacleDetection, occupancyGrid, safePath, distanceMap, navigation, telemetry) through the shared useStageRun + ConsoleResultModal.
- §22.1: all bottom-right .exe buttons removed — the per-window ? buttons now cover every stage; PipelineRunButton, the bespoke runResult modal and useTelemetryRunner were deleted.
- §22.5: the topbar Run Algorithms button kept its name but now runs all tabs sequentially with a pass/fail summary toast and a 'Running...' state.

Sidebars, checklists & selection

- §20.3: Data-tab sidebar got collapsible window subtab groups under 'Tabs to include in report'; the 'All Windows' tile gallery was removed.
- §21: sub-heading rows became checkboxes (see report UX in §5).
- §22.2: Telemetry tab 'Refresh Logs' sidebar shows Telecommand / Telemetry headings with TC1–3 / TM1–3 checkboxes that filter rows 1–3 of each log table (rows beyond 3 always shown).
- §22.3: Data sidebar made identical in style — every possible window sub-heading is always listed under each tab heading (WINDOW_ITEMS module), master tab checkbox kept, collapse chevron removed.
- §25: heading/sub-heading checkboxes show a pure-CSS tick (?) with no fill when selected — .chk--on is white with an accent border, tick via .chk--on::after; both Checkbox components dropped their lucide icon and the unused import.

Tab & screen work

- §7: the new Telemetry & Telecommand tab (log tables + run buttons), Data tab 'All Windows' gallery, and Scene Analysis scroll history (prev/Next/Latest through past query/response entries).
- §22.4: the Telemetry page-level 'Telemetry & Telecommand' section title was removed (table panel titles kept).
- §27: the 'Forgot password?' link is gone from the login screen (commented out in LoginForm.jsx) — the front-end no longer mentions the feature at all; the forgot/reset views stay in the source but are unreachable, and the backend endpoints are untouched.
- §18.2: choosing a workspace folder now auto-loads the data (previously it sat empty until Run bumped the signals).
- §20.1: fullscreen blank white band fixed — .vns uses height:100vh; min-height:660px; box-sizing:border-box so it fills its container in both theme blocks.

Codebase health (§15)

A full optimization scan removed ~265 lines of genuinely dead code (unreferenced overlay/chart components, an unreachable expandedContent.chart branch, unused params, a no-op spinner) and wrapped an IMAGE_MAP in useMemo. Two audit items (navProps, rover state) were re-investigated and KEPT — they are live editable UI state. ESLint baseline went from 19 to 9, all pre-existing set-state-in-effect in the loadAll pattern; no new errors from any later work.

Save-As & share

- §14: native Save dialog for every report with a pre-filled default filename and location

(15_Report/<session>/).

§13: the Data tab's Share button was removed — the sidebar checkboxes feed the report only; share remains on other tabs.

Feature Status

Feature ledger captured in CHAT_LOG §10 — every requested feature is Done:

#	Feature	Status
1	Run Detection button (Obs. Det.)	Done
2	ScriptRunner + stage scripts	Done
3	Forgot / recover password	Done
4	Admin approval for new users	Done
5	Telemetry & Telecommand tab	Done
6	Data tab 'All Windows' gallery	Done
7	Scene Analysis scroll history	Done
8	Per-tab report title alignment	Done
9	Report table layout (Property_01..05)	Done
10	Pipeline Run button on every tab	Done
11	Navigation script (09_Navigation)	Done
12	Scene Analysis script (14_Report)	Done
13	.exe swap support (SCRIPT_CONFIG)	Done
14	Data tab — tab-level report summary	Done
15	Save-As dialog for report generation	Done
16	Nav rail reorder (Data at end)	Done
17	Data-tab sub-heading checkboxes in report	Done
18	Checkbox tick style (§25)	Done
19	Single-file portable deployment exe (§27)	Done
20	Embedded SQLite backend, no MySQL (§27)	Done
21	Forgot-password link hidden in UI (§27)	Done

Packaging, Installer & Delivery

Single-file deployment (§27) — the deliverable

npm run electron:build / npx electron-builder --win portable produces release\vns-app 0.0.0.exe — ONE portable file, 100,499,340 bytes (~96 MB).

Inside it: dist/ (React UI) in app.asar, electron/server/ (embedded Express + SQLite backend), and all 10 stage exes + the better-sqlite3 native module in app.asar.unpacked.

Restored from git history (git checkout e7b182d -- ...) — zero merge conflicts because none of those files had changed since the §19 revert; only build.win.target=["portable"] was added on top.

Machine notes: npm 12 blocks install scripts until approved (npm install-scripts approve

better-sqlite3 electron sharp), the native module is rebuilt for Electron (npx @electron/rebuild), and electron-builder's existing winCodeSign cache means no admin rights are needed to build. QA on the built exe: launches, backend up on :4000 (~30 s first run), admin login + /me session OK, %APPDATA%\vns-app\vns.db seeded (plain SQLite, header 'SQLite format 3'), UI signup/approve flow exercised, all 10 stage exes present, 'Forgot password?' button text absent from the bundle.

Installer & packaged-app fixes (§18)

npm run electron:build produces an NSIS one-click installer (release\vns-app Setup 0.0.0.exe, ~184 MB) plus a portable win-unpacked folder; build/icon.ico regenerated as a real 16/32/48/256 multi-size ICO; package.json description/author stamped.

18.1 blank window in the packaged app — Vite absolute asset paths (/assets/...) broke under file:///; fixed with base: '/' in vite.config.js.

18.3 stage-exe buttons ENOENT — scriptDirs() now checks the real app.asar.unpacked/scripts first in packaged mode (the packed asar path is phantom on disk) and only falls back to the plain path in dev.

Build notes for this machine: electron-builder needs admin for winCodeSign cache extraction (symlink privilege), and a running win-unpacked app locks DLLs and breaks rebuilds (close it / delete release\ first).

.git / delivery

The repo lives at github.com/thegnssproject-beep/VNS-App (branch main). Representative pushed commits: a9c9394 (packaged fixes), 3df2cdd (revert of self-containment), bdb376f (report layout + §22 checklists/Run-All + §25 ticks), bcdf1d2 (CHAT_LOG §25 doc).

RAR password (CHAT_LOG §6): the Vision_Navigation_Software_02.rar (14 MiB, RAR5, password-encrypted) was resolved — the correct password is DHA_SUFA (SUFA, not SUFFA); extracted 44 folders / 103 files into C:\Users\sahibalaljee\Downloads\ (7-Zip, no errors).

Known Issues & Gotchas

No workspace folder on a fresh launch -> stages report 'no root' until the user clicks Select Folder on the Input tab — by design (folder is not persisted across restarts).

electron/main.cjs does NOT hot-reload — changing it requires an app restart (close it, then run start-dev.ps1).

.py can't be spawned directly on Windows — ScriptRunner prepends python; .exe stages need no interpreter.

rg is not installed on this machine — use Select-String / grep in PowerShell.

Vite binds IPv6 ::1 — Test-NetConnection against 127.0.0.1:5173 returns False, but http://localhost:5173 returns 200.

PowerShell 5.1 parses .ps1 without a UTF-8 BOM as ANSI — keep start-dev.ps1 / stop-dev.ps1 ASCII-only with a trailing newline.

NEVER use PowerShell Get-Content/Set-Content on UTF-8 source files (JSX/CJS without BOM) — PS 5.1 reads them as ANSI and mangles em-dashes (mojibake) or injects a BOM. Use the Read/Edit/Write tools; a recovery trick is to encode the bytes as cp1252 and write them raw.

navigation.py writes placeholder rover properties and scene_analysis.py writes canned responses (not AI) — real odometry/AI would need proper backends.

Current Work State & Recent Changes

State as of CHAT_LOG §27 (current)

Deliverable built and QA'd: release\vns-app 0.0.0.exe (portable, ~96 MB) — frontend + backend + database in one file for single-system deployment.

Self-contained architecture (embedded Express + SQLite) restored from e7b182d and adopted as

the shipping one; start-dev.ps1/stop-dev.ps1 no longer touch MySQL; server/ kept in the repo but unused.

'Forgot password?' link hidden from the login screen (commented out in LoginForm.jsx); backend endpoints untouched; button text confirmed absent from the built bundle.

Report = exact ditto of the user's Report Layout.docx (§24); Data + Telemetry sidebars have permanent sub-heading checklists with CSS ? ticks; Run All runs every tab headlessly.

ESLint = 10 pre-existing set-state-in-effect errors only; npm run build + node --check on all touched electron files pass.

Known open thread

The telemetry log can look empty right after an app restart until the workspace folder is re-picked — the data lives on disk (top-level 12_Telemetry_Data + session 11_Telecommand_Data) and reloads once a root is selected.

Delivered via GitHub Release v0.1.0 (latest): the portable exe is attached for direct download; the stale v0.0.0 release/tag from the §19 experiment were deleted. CHAT_LOG/README/report refreshed (§27.1); everything pushed as 1436764 + 1f4f3b2.

This report was generated from CHAT_LOG.md to keep a single source of truth for resuming work later.